

CHAPTER 08

視差滾動 (PARALLAX SCROLLING)

複習：上週重點

- [x] 學會 Cinemachine 攝影機。
- [x] 設定了 Follow 讓相機跟隨主角。
- [x] 設定了 Confiner 限制邊界。

現在攝影機會動了，但你是否覺得背景看起來「平平的」？
像是貼在鏡頭上的一張紙？

本章目標

我們要為 2D 遊戲增加深度感 (Depth)。

1. 理解 **視差 (Parallax)** 的原理。
2. 準備多層背景素材。
3. 撰寫 `Parallax.cs` 腳本。
4. 實現無限循環背景。

注意：我們還沒教角色移動程式，本週測試時請用滑鼠拖曳主角，觀察攝影機與背景的變化。

什麼是視差？

請想像你坐在火車上看窗外：

1. **近處的樹木**：飛快地往後退。
2. **遠處的山**：慢慢地往後退。
3. **更遠的月亮**：幾乎不動。

不同的移動速度，造就了「距離感」。
這就是視差滾動。

準備素材

我們需要把它分層 (由遠到近)：

1. Sky (天空)：最遠，幾乎不動。
2. Mountains (山)：中景，動得慢。
3. Trees (樹)：近景，動得快。
4. Ground (地板)：最快 (跟隨攝影機速度)。

實作步驟 1：場景佈置

1. 把背景圖片拉入場景。
2. 設定好 **Sorting Layer** (Background, Midground)。
3. **注意**：圖片寬度要夠寬，最好是左右可以無縫拼接 (Seamless) 的圖。
4. 將圖片排列成左、中、右三張 (或是使用 Tiled 模式)，確保畫面填滿。

實作步驟 2：視差原理 (數學)

我們不需要真的去移動背景，我們只需要算「相對位移」。

- CamPos: 攝影機位置。
- ParallaxEffect: 視差係數 (0~1)。
 - 1 = 背景完全跟著攝影機走 (看起來像貼在鏡頭上，無限遠)。
 - 0 = 背景不動 (看起來像在同一平面)。
 - 0.5 = 移動速度是攝影機的一半 (遠景)。

實作步驟 3：撰寫腳本

建立腳本 `Parallax.cs`：

```
using UnityEngine;

public class Parallax : MonoBehaviour
{
    private float length, startpos;
    public GameObject cam;
    public float parallaxEffect;

    void Start()
    {
        startpos = transform.position.x;
        length = GetComponent().bounds.size.x;
    }

    void Update()
    {
        // ... (下一頁)
    }
}
```

實作步驟 4：核心邏輯

在 `Update()` 或 `FixedUpdate()` 中：

```
void FixedUpdate()
{
    // 算出背景相對於攝影機的距離（移動）
    float dist = (cam.transform.position.x * parallaxEffect);

    // 更新背景位置
    transform.position = new Vector3(startpos + dist, transform.position.y, transform.position.z);
}
```

這段程式碼會讓背景根據 `parallaxEffect` 跟隨攝影機。

實作步驟 5：無限循環 (LOOP)

如果攝影機一直走，背景圖走完了怎麼辦？
我們需要「瞬移」背景。

```
// 算出臨時位置，用來判斷是否超出邊界
float temp = (cam.transform.position.x * (1 - parallaxEffect));

// 如果超出右邊界，把起點往右移
if (temp > startpos + length) startpos += length;
// 如果超出左邊界，把起點往左移
else if (temp < startpos - length) startpos -= length;
```

(這段邏輯比較抽象，建議直接套用並觀察效果)

設定參數

1. 把 `Parallax.cs` 掛在背景圖片 (GameObject) 上。
2. Cam：把 `Main Camera` 拖進去。
3. Parallax Effect：
 - 天空：設為 `1` (或 0.9)，讓它幾乎跟著相機動。
 - 遠山：設為 `0.5`。
 - 近樹：設為 `0.1` 或是 `0` (如果不需視差)。

測試方法 (重要！)

因為我們還沒寫角色移動程式 (下半學期才會教)。

1. 按下 **Play**。
2. 在 Hierarchy 選取 **Player**。
3. 選取 **Move Tool (W)**。
4. 在 Scene 視窗中，左右拖動 Player。
5. 觀察 Cinemachine 跟隨 Player -> 帶動 Camera -> 帶動 Background。
6. 如果你看到背景層次分明地移動，就成功了！

常見錯誤

Q: 背景動得比主角還快，頭好暈！

A: 你的 `parallaxEffect` 設成負值或是大於 1 了？或是公式寫反了。

Q: 背景圖之間有縫隙？

A:

1. 檢查圖片 Import Settings -> Filter Mode -> 改為 Point (若是像素風)。
2. 檢查 Compression -> None。
3. 確保 Sprite 本身是無縫的 (左右邊緣可以接合)。

Q: 無限循環失敗，背景直接消失？

A: 檢查 `length` 的計算，確認 Sprite Renderer 有正確抓到圖片寬度。

進階技巧：Y 軸視差

如果你的遊戲是垂直卷軸 (往上跳)，那你也需要 Y 軸的視差。

邏輯是一樣的，只是把 `x` 改成 `y`。

但在一般的平台遊戲 (Platformer)，通常只需要 X 軸視差就很有感覺了。

總結：上半學期回顧

恭喜！你已經完成了遊戲的「基礎建設」：

1. Ch01-02: 環境建置與介面。
2. Ch03: Tilemap 畫地圖。
3. Ch04: C# 程式基礎。
4. Ch05-06: 物理與互動 (金幣、陷阱)。
5. Ch07-08: 攝影機與視差背景。

現在你有一個**畫面精美、物理真實、有互動機制**的世界。
只差一個能動的主角了！

還是視差滾動的程式碼看不懂？